---

name: Minimal Viral Perch MVP overview: "Ship a minimal, viral wedge for small startups (especially AI apps): Git-backed stack auto-detection + a secure hosted cockpit for team-shared health/logs/context, with frictionless node/edge editing (CLI + hosted UI via PRs), without becoming a feature dump." todos:

- id: inventory-current-demo-path content: Select and harden one public example repo path as the flagship demo (init → TUI/viz → status → logs). status: pending
- id: init-autodetect-polish content: Make `perch init` reliably auto-detect a small provider set from committed files and generate a working `perch.yaml` in <60s. status: pending
- id: hosted-team-cockpit content: Build a minimal hosted GUI (auth + repo connect) that renders the graph and shows status/logs/context as a shared team view. status: pending
- id: real-health-v1 content: Implement real health checks for the v1 providers (not placeholders) and surface red/amber/green in TUI + `perch status --json`. status: pending
- id: logs-command content: Add `perch logs --node <name>` (or equivalent) to stream one node's logs for v1 providers. status: pending
- id: hosted-edit-via-pr content: Add hosted GUI editing for nodes/edges that proposes changes via PR (Git model), preserving review + versioning. status: pending
- id: agent-context content: Ensure `perch context --for-agent` is high-signal for debugging AI app incidents (endpoints, provider URLs, health, last errors) with safe redaction. status: pending
- id: ai-app-provider-pack content: Ship the v1 "AI startup core 6" provider set (Vercel, Postgres, Supabase, OpenAI, Anthropic, Pinecone) end-to-end (detect + dashboard link + meaningful health), plus a v1.5 add-on list (Sentry, Upstash, Stripe). status: pending isProject: false

---

Product thesis (minimal effective product)

Perch's smallest "viral" wedge is **stack situational awareness** for modern startup repos (especially AI apps): *in one place, right now, is my app okay?*

- **Frictionless**: run `perch init` in a repo and immediately get a working view.
- **Unique value**: Perch auto-detects your stack from committed files and shows a **single, navigable graph** with **real health** and **one-node logs**.
- **Natural adoption distribution**: the hosted cockpit becomes the team's default "shared context" for incidents and debugging ("link the Perch view for this service"), and the Git-backed config makes changes reviewable and safe.

This avoids a feature dump by hard-scoping to **one outcome**: "I can tell if my app is okay,

and if not, where to look next."

Why this fits what you already built

You already have the core primitives needed for this wedge:

- **Single binary CLI + TUI**: [cmd/perch/main.go](/Users/yashgupta/Desktop/Programming/perch/cmd/perch/main.go), [internal/cli/root.go](/Users/yashgupta/Desktop/Programming/perch/internal/cli/root.go)
- **Repo-local config contract** (`perch.yaml`): discovery + parsing in [internal/config/resolve.go](/Users/yashgupta/Desktop/Programming/perch/internal/config/resolve.go), schema in [internal/config/config.go](/Users/yashgupta/Desktop/Programming/perch/internal/config/config.go)
- **Graph + context output** (already agent-friendly): [internal/cli/context.go](/Users/yashgupta/Desktop/Programming/perch/internal/cli/context.go), [internal/stackcontext/assemble.go](/Users/yashgupta/Desktop/Programming/perch/internal/stackcontext/assemble.go)
- **Web visualization server**: [internal/cli/viz.go](/Users/yashgupta/Desktop/Programming/perch/internal/cli/viz.go) serving SPA embedded by [web/embed.go](/Users/yashgupta/Desktop/Programming/perch/web/embed.go)
- **Provider catalog system** (including Vercel spec): [providers/embed.go](/Users/yashgupta/Desktop/Programming/perch/providers/embed.go), [providers/hosting/vercel.yaml](/Users/yashgupta/Desktop/Programming/perch/providers/hosting/vercel.yaml)
- **Work-in-progress but valuable**: env scanning + auth exists in [worktrees/env-scanning/](/Users/yashgupta/Desktop/Programming/perch/worktrees/env-scanning) and can be upstreamed selectively.

MVP definition (what we ship)

User story

"As a startup engineer shipping an AI app, I want to open one tool and immediately see whether my stack is healthy, with one keystroke to pull logs/context for the broken component, so I don't lose days of momentum to blind debugging."

Hard scope boundaries (to prevent feature dump)

- **In scope**:
  - `perch init`: auto-detect a small, opinionated provider set from committed files and

    - `perch init`: auto-detect a small, opinionated provider set from committed files and scaffold a usable `perch.yaml`.
    - `perch` TUI: graph with **colored health** (green/amber/red), keyboard navigation, and fast refresh.
    - `perch status`: one-command health check with `--json` for automation/agents.
    - `perch logs --node X`: stream logs for a single node (for providers that support it in v1).
    - `perch context --for-agent`: high-signal, redacted context for "what to do next" when something is unhealthy.
    - **Graph editing (minimal, both surfaces)**:\n+ - CLI/TUI: quick add/remove edges (and minimal node edits)\n+ - Hosted: add/delete/replace nodes+edges via a form UI that **opens a PR** with `perch.yaml` changes (Git model)\n+ - **Team shared context (hosted, secure)**:\n+ - auth + workspace\n+ - connect a repo and render the current graph\n+ - shareable permalinks to a node/status view (access-controlled)
    - "AI app support" (v1): health for common AI dependencies (LLM provider connectivity, vector DB reachable, Postgres reachable) and basic latency/error signals where available.

- **Explicitly out of scope (v1)**:
    - PR preview automation, PR comments, or eval harnesses (can be v2 once the core loop is loved)
    - Snapshots/rollback/timeline/blame
    - Marketplace/registry and multi-provider deployment pipelines
    - Full observability (metrics/traces dashboards) and enterprise auth/RBAC
    - Non-Git config collaboration (no "stateful" hosted config editor that diverges from repo)

Key product design choices (with tradeoffs)

- **Keep `perch.yaml` as the contract**: it makes detection/scaffolding reversible and keeps teams in control.
- **Health + logs before "actions"**: Perch earns trust by being a reliable cockpit before it tries to mutate infra.
- **AI-app support means dependency awareness**: focus on "is the model/vector DB/DB reachable and behaving" rather than building an eval product.
- **Git model for team collaboration**: repo remains source-of-truth; hosted UI proposes edits via PRs for versioning/review and easy future GitHub integrations.

Rollbacks policy (v1)

- **Hard rule**: no "future" rollback previews or UI hints.

- **V1 stance**: **no service deploy rollbacks in v1**.
  - If a provider rollback is not implemented, the UI does not mention it.
  - The only "rollback" in v1 is **Git rollback of `perch.yaml`** via normal PR/commit reverts.
- **V2 stance**: rollbacks are **provider-scoped and explicit** (e.g., Vercel only), and only appear in UI when implemented and permissioned.

Implementation plan (reuse-first)

1) Make the existing local story smooth (day 1–3)

- Ensure `go build` works from a clean checkout by documenting/building `web/dist` needs (since `[web/embed.go](/Users/yashgupta/Desktop/Programming/perch/web/embed.go)` embeds `dist`).
- Pick a single runnable public example as the flagship demo (likely `[examples/scenarios/full-stack/](/Users/yashgupta/Desktop/Programming/perch/examples/scenarios/full-stack)`), and ensure:
  - `perch init`
  - `perch viz`
  - `perch context --for-agent` all work end-to-end.

2) Tighten the "viral loop": init → graph → status (day 3–7)

- Ensure `perch init` reliably detects and scaffolds a working stack for a small provider set.
- Replace placeholder health for "deployable" providers with real v1 health checks (even if minimal: API reachability + project exists + last deploy status).
- Make the TUI "GIF-ready": fast refresh, stable layout, clear colors, and helpful node details.

3) Add `perch logs --node X` for v1 providers (day 7–12)

- Implement log streaming where the provider supports it (start with one provider to keep scope tight).
- Fallback behavior: when logs aren't supported, show a clear message and point to the provider's dashboard URL.

4) Upstream env scanning for frictionless setup (day 10–14)

- Pull in the best pieces from `[worktrees/env-scanning/](/Users/yashgupta/Desktop/Programming/perch/worktrees/env-scanning)`:
  - detect referenced env vars and generate `.env.example` deltas
  - classify "safe for preview" vs "must be local/prod only" (starter heuristics)

- Add `perch doctor` (or extend `perch init`) to print a single “what’s missing / what’s unsafe” checklist.

5) “AI app starter pack” provider focus (day 12–20)

- Define and ship a tight provider set where Perch can go end-to-end (detect → dashboard link → meaningful health state), optimized for AI startups. \n+#### V1 provider set (AI startup core 6) \n+- **Vercel** (`vercel`)\n+ - detect: `vercel.json`\n+ - v1 health: project reachable + last deploy status (minimal but real)\n+- **Postgres** (`postgres`)\n+ - detect: `prisma/schema.prisma` provider=postgres OR `pg` dependency\n+ - v1 health: TCP/connect probe (strong “is it up?” signal)\n+- **Supabase** (`supabase`)\n+ - detect: `@supabase/supabase-js`\n+ - v1 health: API reachability + project exists (minimal but real)\n+- **OpenAI** (`openai`)\n+ - detect: `openai` or `@langchain/openai`\n+ - v1 health: API reachability + auth validity (no cost-heavy calls)\n+- **Anthropic** (`anthropic`)\n+ - detect: `@anthropic-ai/sdk`\n+ - v1 health: API reachability + auth validity\n+- **Pinecone** (`pinecone`)\n+ - detect: `@pinecone-database/pinecone`\n+ - v1 health: control-plane reachability + cheap “list indexes” (if available) or equivalent\n+\n+#### V1.5 add-ons (after core is solid)\n+\n+- **Sentry** (`sentry`)\n+- **Upstash** (`upstash`)\n+- **Stripe** (`stripe`)\n+\n+#### Provider quality guardrails (anti feature-dump)\n+\n+- **No half-wired providers**: a provider is only “shipped” when it supports:\n+ - detection OR explicit config\n+ - dashboard link\n+ - meaningful health (no always-green stubs)\n+- **Missing credentials is amber**: if a provider needs an API key and it’s not present, show “needs credentials” (amber) rather than red/green.\n+\n+For each provider, define v1 health signals + dashboard URLs + basic metadata in `context --for-agent`.

6) Launch packaging (day 18–24)

- Make install friction tiny: `brew install` / curl script, plus a 60-second quickstart.
- Create 1–2 “wow” demo repos that reliably render the green graph + show an unhealthy flip + logs drill-down.
- Minimal hosted onboarding:\n+ - sign in\n+ - connect GitHub repo\n+ - see graph + share a secure link to a node view\n+ - edit graph → open PR

Architecture sketch

```
flowchart LR
  DevRepo[DevRepo] --> PerchCLI[PerchCLI]
  PerchCLI --> PerchYAML[perchYaml]
  PerchCLI --> ProviderDetect[ProviderDetect]
  PerchCLI --> StatusCollector[StatusCollector]
  PerchCLI --> LogsCollector[LogsCollector]
  PerchCLI --> VizServer[VizServer]
  ProviderDetect --> PerchYAML
  StatusCollector --> Health[HealthStatus]
```

```
  LogsCollector --> Logs[LogsStream]
  Health --> VizServer
  DevRepo --> HostedGUI[HostedGUICockpit]
  HostedGUI -->|"reads perchYaml from git"| PerchYAML
  HostedGUI -->|"writes changes via PR"| GitHubPR[GitHubPullRequest]
  HostedGUI --> Health
  HostedGUI --> Logs
```

“Definition of done” for initial launch

- A new startup repo can do:
  - `perch init` (or copy-paste a tiny starter `perch.yaml`)
  - `perch` (TUI) or `perch viz` to see the graph
  - `perch status --json`
  - `perch logs --node <name>` for at least one v1 provider
  - `perch context --for-agent` for a broken node
- The “viral moment” exists:
  - `perch init` correctly detects 3–5 common services from committed files
  - the graph shows meaningful health (not placeholders)
  - a screenshot/GIF tells a clear story without explanation

Metrics to validate ‘minimal effective’

- Time from clone → first working graph: **< 60 seconds**
- Time to answer “what’s broken?”: **< 30 seconds** (open Perch → see red node → open node detail/logs)
- Share rate: screenshots/GIFs; stars; “try this” mentions in Slack/Twitter

Git workflow to conserve work and keep shipping

Goals

- Keep `main` always buildable and demoable (you can cut a release anytime).
- Preserve and gradually upstream existing work (notably under `worktrees/`) without a “big bang merge.”
- Support parallel workstreams (CLI/TUI, hosted GUI, provider runtimes, env scanning) with small PRs.

Recommended model: trunk-based with short-lived feature branches + a release branch

- **`main`**: always green; only merge via PR; no long-running WIP merges.

- **Feature branches**: small, scoped, short-lived; behind flags if needed.
- **`release/*` branch**: cut when you're ready to ship; only accept bugfixes and polish.

How to handle the existing `worktrees/*` code

The `worktrees/` directories are valuable prototypes but risky to "merge wholesale." Treat them as donor branches:

- **Create a tracking issue per worktree**:
  - `env-scanning` donor: upstream only the minimal env detection that reduces setup friction.
  - `full-platform` donor: upstream only provider runtime helpers that unblock v1 health/logs.
- **Cherry-pick by capability, not by folder**:
  - Identify 1–3 concrete capabilities to extract.
  - Land them as small PRs into `main`, adjusting paths to match the root layout.

Explicit cleanup decision (to prevent confusion and divergence):

- After the needed capabilities are upstreamed, **convert each `worktrees/<name>` snapshot into a real git branch** (for historical reference), then **remove the `worktrees/` directories from `main`**.
- Rule: `main` should not contain duplicated implementations split between root and `worktrees/`.

Branch / PR discipline (guardrails against feature creep)

- Every PR must map to exactly one v1 checkbox (health, logs, init detect, hosted cockpit, PR-based editing, incident packet).
- No PR may introduce a "new category" of feature (e.g., rollbacks, tracing, evals) without an explicit v2 label.
- Prefer "vertical slices" over refactors: one provider end-to-end health + logs beats 3 half-wired providers.

Release cadence

- Weekly internal release (tagged) to keep the demo always improving.
- Public launch when the "definition of done" bullets are met and the hosted cockpit onboarding is smooth